Atoms Demo 说明文档

小工具生成器（Mini-Tool Generator）· 一个能生成应用并持续改的 Agent Demo

ROOT / AI Native 全栈工程师 · 笔试交付材料

> **一句话概括：输入一句自然语言需求，智能体生成一个可直接运行的网页小工具；**
>
> **支持 3 路并发“赛马”择优、在原页面上连续修改，生成结果持久化，刷新或换浏览器仍在。**

提交人	_____（提交前填写姓名）
提交日期	2026-10-29
Demo 在线地址	http://8.221.136.192:8000
GitHub 代码仓库	https://github.com/xueerhuan2014/atoms-toolgen（public，已核验）
技术栈	Python 3 / FastAPI / Uvicorn / httpx；原生 HTML + CSS + JavaScript（无框架、无构建）；DeepSeek（OpenAI 兼容接口）；Supabase（PostgREST）；阿里云服务器 + systemd（端口 8000）
本地启动	pip install -r requirements.txt → 复制 .env.example 为 .env（可全空，自动进入 mock 模式）→ python -m uvicorn main:app --host 0.0.0.0 --port 8000

说明：Demo 已常驻部署在阿里云服务器（systemd 托管，端口 8000），开机自启、进程异常自动拉起，链接长期有效；仓库 README 仍保留 Cloudflare 隧道与 render.yaml 作为备用方案。评审建议使用 PC 端 Chrome / Edge 访问，PC 端全部功能正常；手机端仅作辅助体验。

一、 题目理解与最小闭环

这道题考察的不是“用 AI 做了一个业务应用”，而是“做一个能生成应用的 AI Agent 平台”。因此产物必须是可运行的网页应用本身，而不是聊天记录、代码片段或静态截图。我据此锁定了四环闭环：

① 一句话输入 → ② 智能体处理、生成代码 → ③ 页面内直接渲染运行 → ④ 结果持久化（刷新 / 换浏览器仍在）

自检标准：把生成出来的 HTML 单独存成 .html 文件、双击用浏览器打开，仍然能打开、能点、能算，才算没有跑题。

- **选题：**小工具生成器。理由：需求最具体、生成最稳定、演示最不容易翻车，并且天然适合展示“赛马模式”这个亮点。

输入“一个能算房贷月供、带提前还款对比的计算器”，即生成一个可直接使用的计算器页面。

二、 实现思路与数据流

用户输入（一句话需求）

```
        │  POST /api/generate  或  /api/generate_race
        ▼
```

FastAPI 后端（main.py）

① 组装 Prompt：强制单文件 HTML、CSS/JS 内联、无外部依赖、不带 Markdown 围栏
② 调用大模型（OpenAI 兼容 /chat/completions，超时 60s，失败退避重试 2 次）
③ 清洗输出：剥离代码围栏与解释文字，只留完整 HTML
④ 失败 / 无 Key → 回退 3 个真实可用的 mock 示例（保命层）

```
        │
        ▼
```

单文件 HTML → 前端 iframe（Blob URL）实时渲染，可直接交互

```
        │
        ▼
```

Supabase（projects / messages 两张表）持久化

```
        │
        ▼
```

历史列表 → 重新打开 → 连续修改（当前 HTML + 新要求 → 写回同一项目）

2.1 单路生成（核心闭环）

前端把需求 POST 到 /api/generate；后端组装 system + user 提示词（要求模型只输出一段完整、自包含、可直接在浏览器运行的单文件 HTML，CSS/JS 全内联、不要外部依赖、不要 Markdown 代码块包裹），调用 OpenAI 兼容接口后清洗掉可能的围栏，再把 HTML 交给前端。前端用 Blob URL 塞进 iframe，生成物立刻可点可用。

2.2 赛马择优（亮点能力）

同一需求用 asyncio.gather 并发 3 路生成，温度分别为 0.4 / 0.7 / 1.0，并附加“简洁实用 / 美观现代 / 创意大胆”的风格约束。单路异常被隔离（return_exceptions + 单路兜底），失败只占一格并显示原因，其余路正常渲染。用户点「就用这个」后，把选中版本回填到生成时已创建的同一条项目记录，避免历史里出现重复项。

2.3 连续修改（在原应用上迭代）

打开任意历史项目后输入修改要求，后端把“当前完整 HTML + 新要求”一起发给模型，temperature 设为 0.4 降低跑偏，并要求只做最小必要改动。成功才写回 projects 并追加 messages；失败返回 502 可读提示，原版本内容保持不变。

2.4 持久化与兜底

- **持久化**：projects(id, prompt, final_html, race_versions, created_at) 与 messages(id, project_id, role, content, created_at)。后端用 service_role key 走 PostgREST，RLS 开启但 service_role 绕过，anon 无法直接读写；前端不接触任何密钥。
- **保命层**：无 Key / Key 无效 / 额度耗尽 / 超时 / 5xx 时自动回退 3 个预置且真实可用的示例（房贷计算器、待办清单、JSON 格式化），并返回 notice 说明原因；接口仍返回 200，评委任何时候打开都能看到东西。
- **健壮性**：参数校验、HTTP 异常、未捕获异常统一转成 {error, status, message} 中文 JSON；前端统一 loading / toast，并监听 unhandledrejection，页面不白屏。
- **并发细节**：同步模型调用放入 asyncio.to_thread，避免阻塞事件循环导致健康检查、历史列表无响应。

三、 关键取舍：为什么这么选，砍掉了什么

决策点	选择	放弃的方案	理由与代价
选题	小工具生成器	待办看板 / 个人主页生成器	需求边界最清晰、生成最稳、演示最不容易翻车；代价是炫技空间较小，靠赛马与连续修改补亮点。

决策点	选择	放弃的方案	理由与代价
产物形态	单文件自包含 HTML	多文件工程（React / Vite）、带后端代码	iframe 可直接运行、数据库只需存一个字段、无构建链；代价是复杂应用无法覆盖。
前端	原生 HTML/CSS/JS 单页	React / Next.js	零构建、零依赖、部署最简单，单个文件即可完整审阅；代价是代码量偏大、无组件复用。
后端	FastAPI + Uvicorn	Node / Express、Next API	异步并发写赛马很轻（asyncio.gather），参数校验与异常处理开箱即用，Render 部署成熟。
持久化	Supabase（Postgres + PostgREST）	SQLite 落本地磁盘	免费实例磁盘是临时的，重启即清空，是避坑指南点名的扣分重灾区；代价是多一个外部依赖与配置步骤。
模型接入	OpenAI 兼容协议（当前 DeepSeek）	绑定单一厂商 SDK	换供应商只改环境变量（Base URL / Model），少一个依赖；代价是需自己做输出清洗。
赛马路数	3 路并发	5 路以上	额度与耗时可控（并发耗时≈单路），足够体现“多方案择优”；代价是多样性有限。
公网交付	阿里云服务器 + systemd 常驻	Render 免费实例 / Cloudflare 隧道	链接固定、开机自启、进程异常自动拉起，不受本机关机影响；代价是需要一台服务器并放行端口。
无 Key 策略	内置 3 个真实可用 mock	直接报错 / 只放静态图	保证额度归零、断网、超时也能完整演示；代价是 mock 结果与用户需求不完全对应。

四、 AI 工具的使用方式：AI 写代码，人守关键节点

工具链：Trae（Builder / Agent 模式）+ 内置终端。AI 负责生成与修改代码，任务拆解、技术取舍、部署与最终验收由我判断，关键节点不交给 AI 自行判定。

- **阶段化会话**：骨架 → 接真实模型 → 赛马 → 持久化 → 健壮性 → 文档，每个阶段新开会话，避免上下文越滚越贵、越改越偏。

- **给边界：**每次改动都明确边界，例如“只改 generate_app 与 /api/generate 对应部分，保留 mock，不重写前端”，避免 AI 顺手重写整个项目。
- **先 mock 后真调：**先用预置示例跑通“输入 → 生成 → 渲染 → 落库”整条链路，再换真实 Key，避免把调试成本烧在模型调用上（额度纪律）。
- **人工验收点：**① 生成物脱离平台单独打开仍能运行；② 无 Key / 错 Key / 空输入 / 断网都不崩；③ 刷新与换浏览器后历史仍在；④ 仓库 public 且搜不到密钥；⑤ 公网链接用无痕 + 手机流量各开一次。

五、 当前完成程度

5.1 已实现

能力	实现要点	验收方式
一句话生成	/api/generate → 大模型 → 清洗 → 返回完整 HTML	输入“房贷计算器”，iframe 出现可输入、可计算、可交互的页面
赛马择优	3 路 asyncio 并发，温度 0.4 / 0.7 / 1.0 + 风格提示；单路失败隔离	三栏并排对比，点「就用这个」高亮选中并落库
连续修改	/api/projects/{id}/iterate，原 HTML + 新要求，temperature 0.4，失败不写回	输入“把主色调改成蓝色”，页面在原结果上更新
数据持久化	Supabase projects / messages，service_role 仅后端持有	刷新页面、换浏览器后历史仍在，可重新打开复现
兜底与容错	3 个真实可用 mock；超时 60s / 90s + 退避重试 2 次；401/403 不重试；全局异常 JSON	无 Key、错 Key、断网、空输入均不白屏，流程可继续
体验细节	示例提示词 chips、模式标签、健康检查、刷新 / 新窗口 / 复制源码、手机端抽屉	首次进入就知道能做什么，手机宽度下布局可用
工程与交付	README、.env.example、.gitignore、render.yaml、public 仓库无密钥	无痕窗口可读仓库与 README，代码仅从环境变量读密钥

5.2 未做 / 有意不做

项目	状态	说明
注册 / 登录	未做	题目明确“无需注册功能”，把时间留给核心闭环。
赛马自动打分	未做	当前由用户人工点选；自动评审需要额外模型调用与稳定评分标准，先保证闭环可靠。
流式输出	未做	当前用超时 + 重试保证成功率；流式属于体验优化，列入后续 P1。
版本 diff / 回滚	未做	messages 已记录每次对话与 HTML，尚未做可视化 diff 与一键回滚。
导出 zip / 分享短链	未做	已有“复制源码 / 新窗口打开”，完整导出与只读分享列入后续。
多文件工程生成	未做	刻意约束为单文件 HTML，保证 iframe 内可直接运行；多文件/带后端应用需要沙箱方案。
常驻托管 / 正式域名	部分完成	已用 systemd 常驻托管在阿里云服务器（http://8.221.136.192:8000），开机自启、崩溃自动拉起；自定义域名与 HTTPS 尚未配置。
自动化测试 / CI	未做	时间优先给“能打开、能点、能存”三条硬要求；测试与 CI 列入工程化阶段。

5.3 提交前实测记录（2026-10-29）

检查项	结果
公网首页 http://8.221.136.192:8000	HTTP 200，正常返回前端页面
/api/health	llm_configured = true，supabase_configured = true
历史持久化	/api/projects 可读到已生成记录（3 条），含赛马记录
仓库可见性	GitHub API 返回 visibility = public，无痕可打开
密钥安全	.env 已被 .gitignore 屏蔽；代码仅从环境变量读取；仓库检索无 sk- / token / password 硬编码

检查项	结果
移动端	响应式布局 + 历史侧栏抽屉；提交前建议用手机流量再完整走一遍

六、 如果继续投入时间：扩展方向与优先级

排序原则：先修“会让评委打不开或不敢信”的确定性风险，再把已有亮点做深，最后才是横向扩功能。即：交付确定性 > 核心体验 > 亮点深度 > 功能广度。

优先级	事项	为什么现在做 / 怎么做	预期收益
P0	自定义域名 + HTTPS + 可用性监控	已用 systemd 完成常驻托管；下一步绑定域名、启用 HTTPS，并对 /api/health 做定时探测与告警。	链接更正式、不暴露 IP，长期可用
P0	赛马自动评估与推荐	规则校验（HTML 可解析、必须含交互脚本、需求关键词覆盖）+ 一次低成本 LLM 评审打分，仍保留人工「就用这个」。	把“多路生成”升级为真正的“择优”，最贴 Atoms 的招牌能力
P1	流式输出与阶段进度	用 SSE 逐步推送 HTML，前端增量渲染，同时展示“正在生成 / 正在校验”阶段。	缓解 30–60s 等待，体验提升明显
P1	版本管理与 diff / 回滚	基于 messages 增加 versions 快照表，展示每次修改差异，支持一键回滚上一版。	连续修改可追溯，降低“越改越坏”的风险
P1	导出与只读分享	一键下载单文件 HTML；用短链提供只读快照页。	成本低，传播与交付体验好
P2	模板库与示例提示词	预置 10–20 个高频工具模板，一键套用后继续修改。	降低空输入门槛，提高首次生成成功率
P2	多文件工程生成 + 沙箱	支持 React / 多文件 / 带后端的小应用，使用独立域名与 CSP 隔离预览。	扩展能力边界，但工程量与安全成本高
P2	工程化	结构化日志与错误上报、限流与配额、pytest 覆盖核心接口、GitHub Actions。	长期质量，非演示必需

七、 已知风险与边界（主动交代）

- **常驻部署：线上服务由 systemd 托管在阿里云服务器（/opt/atoms-toolgen，端口 8000），服务器重启或进程异常会自动拉起；Cloudflare 隧道仅作为本地演示的备用方案。**
- **模型与额度：**模型额度耗尽、Key 失效或网络抖动时自动降级 mock，流程不中断；但 mock 与具体需求语义不一定完全匹配，界面会通过 notice / toast 明确说明当前为兜底结果。
- **沙箱隔离：**生成代码运行在 iframe sandbox 中（allow-scripts / allow-forms / allow-same-origin / allow-downloads），对演示足够；生产环境应改为独立域名 + CSP + 去掉 same-origin，避免生成代码触碰父页面。
- **单文件约束：**只支持纯前端可完成的小工具，不能生成需要服务端或多文件的完整应用，这是单文件约束的直接代价。
- **云端依赖：**Supabase 免费项目长期不活跃可能被暂停；表结构极简，迁移与恢复成本低。
- **终端兼容性：**完整功能以 PC 端 Chrome / Edge 为准，生成、赛马、历史记录、连续修改、复制源码 / 新窗口打开均正常；手机端已完成响应式适配，可提交需求并查看结果，但受屏幕尺寸与部分手机浏览器 WebView 对 iframe 支持差异的影响，预览区域偏小、需上下滚动，少数机型可能出现历史记录打开后显示不完整，正式演示建议优先使用 PC 端。

八、 可直接粘贴到【4. 笔试结果回收】

【4. 笔试结果回收】

1）笔试文档：本说明文档（飞书副本链接：_____）

2）Demo 在线地址：http://8.221.136.192:8000

3）GitHub 源码（public）：https://github.com/xueerhuan2014/atoms-toolgen

4）说明文档：即本文档（实现思路与关键取舍 / 当前完成程度 / 继续投入的优先级）

备注：Demo 已常驻部署在阿里云服务器（systemd 托管），链接长期有效；如服务异常会自动拉起。

文档生成时间：2026-10-29 | Demo：http://8.221.136.192:8000 | 仓库：https://github.com/xueerhuan2014/atoms-toolgen